Unit 4 - Preliminaries

Essential reading

Wayner, Peter *Compression Algorithms for Real Programmers*. (Morgan Kaufmann, 2000) [ISBN 0-12-788774-1]. Chapter 2.

Further reading

Salomon, David *A Guide to Data Compression Methods*. (Springer, 2001) [ISBN 0-387-95260-8]. Chapter 1.

Huffman coding

Huffman coding is a successful compression method used originally for text compression. It assumes that each character is stored as a 8-bit ASCII code.

You may have already come across Huffman coding in a programming course. If so, review the following two questions:

1. What property of the text does Huffman coding algorithm require in order to fulfil the compression?
2. What is the main idea of Huffman coding?

Huffman coding works well on a text file for the following reasons:

- Characters are represented normally by fixed-length codewords[1] in computers. The codewords are often 8-bit long. Examples are ASCII code and EBCDIC code.

  Example 3.1 *In ASCII code, codeword* p1000001 *represents character 'A';* p1000010 *'B';* p1000101 'E', *etc., where* p *is the parity bit.*

- In any text, some characters occur far more frequently than others. For example, in English text, letters E,A,O,T are normally used much more frequently than J,Q,X.
- It is possible to construct a *uniquely decodable* code with variable codeword lengths.

[1]*Note: it is useful to distinguish the term 'codeword' from the term 'cord' although the two terms can be exchangeable sometimes. In this subject guide, a code consists of a number of codewords (see Example 3.1.)*

Our aim is to reduce the total number of bits in a sequence of 1s and 0s that represent the characters in a text. In other words, we want to reduce the average number of bits required for each symbol in the text.

Huffman's idea

Instead of using a fixed-length code for each symbol, Huffman's idea is to represent a frequently occurring character in a source with a shorter code and to represent a less frequently occurring one with a longer code. So for a text source of symbols with different frequencies, the total number of bits in this way of representation is, hopefully, significantly reduced. That is to say, the number of bits required for each symbol *on average* is reduced.

Example 3.2 *Frequency of occurrence:*

E	A	O	T	J	Q	X
5	5	5	3	3	2	1

Suppose we find a code that follows Huffman's approach. For example, the most frequently occurring symbol E *and* A *are assigned the shortest 2-bit codeword, and the lest frequently occurring symbol* X *is given a longer 4-bit codeword, and so on, as below:*

E	A	O	T	J	Q	X
10	11	000	010	011	0010	0011

Then the total number of bits required to encode string 'EEETTJX' is only $2 + 2 + 2 + 3 + 3 + 3 + 4 = 19$ *(bits). This is significantly fewer than* $8 \times 7 = 56$ *bits when using the normal 8-bit ASCII code.*

Huffman encoding algorithm

A frequency based coding scheme (algorithm) that follows Huffman's idea is called *Huffman coding*. Huffman coding is a simple algorithm that generates a set of variable-size codewords of the minimum average length. The algorithm for Huffman encoding involves the following steps:

1. Constructing a frequency table *sorted* in descending order.
2. Building a *binary tree*
   Carrying out iterations until completion of a complete binary tree:
   (a) Merge the last two items (which have the minimum frequencies) of the frequency table to form a new combined item with a sum frequency of the two.
   (b) Insert the combined item and update the frequency table.
3. Deriving *Huffman tree*
   Starting at the *root*, trace down to every *leaf*; mark '0' for a *left branch* and '1' for a *right* branch.
4. Generating Huffman code:
   Collecting the *0*s and *1*s for each path from the root to a leaf and assigning a 0-1 codeword for each symbol.

We use the following example to show how the algorithm works:

Example 3.3 *Compress 'BILL BEATS BEN.' (15 characters in total) using the Huffman approach.*

1. **Constructing the frequency table**

B	I	L	E	A	T	S	N	SP(space)	.	character
3	1	2	2	1	1	1	1	2	1	frequency

Sort the table in descending order:

```
B  L  E  SP I  A  T  S  N  .
3  2  2  2  1  1  1  1  1  1
```

2. **Building the binary tree**

There are two stages in each step:

(a) *combine the last two items on the table*

(b) *adjust the position of the combined item on the table so the table remains sorted.*

For our example, we do the following:

(a) *Combine*

```
B  L  E  SP I  A  T  S  (N.)
3  2  2  2  1  1  1  1  2
```

Update[2]

```
B  (N.)  L  E  SP I  A  T  S
3  2     2  2  2  1  1  1  1
```

(b)
```
B  (TS)  (N.)  L  E  SP I  A
3  2     2     2  2  2  1  1
```

(c)
```
B  (IA)  (TS)  (N.)  L  E  SP
3  2     2     2     2  2  2
```

(d)
```
(E SP)  B  (IA)  (TS)  (N.)  L
4       3  2     2     2     2
```

(e)
```
((N.)  L)  (E SP)  B  (IA)  (TS)
4          4       3  2     2
```

(f)
```
((IA)(TS))  ((N.)  L)  (E SP)  B
4           4          4       3
```

(g)
```
((E SP) B)  ((IA)(TS))  ((N.)  L)
7           4           4
```

(h)
```
(((IA)(TS))  ((N.)  L))  ((E SP) B)
8                        7
```

(i)
```
((((IA)(TS))  ((N.)  L))  ((E SP) B))
15
```

The complete binary tree is:

```
    ((((IA)(TS))  ((N.)  L))  ((E SP) B))
                   /                 \
    (((IA)(TS))  ((N.)  L))   ((E SP) B)
         /          \              /    \
 ((IA)(TS))      ((N.)  L)     (E SP)   B
    /   \          /   \        / \
 (IA)  (TS)      (N.)   L      E   SP
  / \   / \      / \
 I  A  T  S     N  .
```

[2]*Note:* (N.) *has the same frequency as* L *and* E*, but we have chosen to place it at the highest possible location - immediately after* B *(frequency 3).*

1. **Deriving Huffman tree**

```
      ((((IA)(TS))  ((N.)    L)) ((E  SP)  B))
                     0/              \1
      (((IA)(TS))  ((N.)    L))   ((E  SP)  B)
          0/          \1              0/      \1
  ((IA)(TS))       ((N.)    L)     (E  SP)    B
    0/     \1        0/     \1       0/ \1
  (IA)   (TS)      (N.)     L        E    SP
  0/ \1  0/ \1     0/ \1
   I  A    T  S     N  .
```

2. **Generating Huffman code**

	I	A	T	S	N	**.**	L	E	SP	B
	0000	0001	0010	0011	0100	0101	011	100	101	11
x	1	1	1	1	1	1	2	2	2	3
	4	4	4	4	4	4	3	3	3	2

3. **Saving percentage**

Comparison of the use of Huffman coding and the use of 8-bit ASCII or EBCDIC Coding:

Huffman	*ASCII/EBCDIC*	*Saving bits*	*Percentage*
48	*120*	*72*	*60%*
		120 − 48 = 72	72/120 = 60%

Decoding algorithm

The decoding process is based on the same Huffman tree. This involves the following types of operations:

- We read the coded message bit by bit. Starting from the root, we follow the bit value to traverse one edge down the the tree.
- If the current bit is 0 we move to the left child, otherwise, to the right child.
- We repeat this process until we reach a leaf. If we reach a leaf, we will decode one character and re-start the traversal from the root.
- Repeat this read-move procedure until the end of the message.

Example 3.4 *Given a Huffman-coded message, 111000100101111000001001000111011100000110110101, what is the decoded message?*

```
      ((((IA)(TS))  ((N.)    L)) ((E  SP)  B))
                     0/              \1
      (((IA)(TS))  ((N.)    L))   ((E  SP)  B)
          0/          \1              0/      \1
  ((IA)(TS))       ((N.)    L)     (E  SP)    B
    0/     \1        0/     \1       0/ \1
  (IA)   (TS)       N.      L        E    SP
  0/ \1  0/ \1     0/ \1
   I  A    T  S     N  .
```

After reading the first two 1s of the coded message, we reach the leaf B. Then the next 3 bits 100 lead us to the leaf E, and so on.

Finally, we get the decoded message: 'BEN BEATS BILL.'

Observation on Huffman coding

1. Huffman codes are not unique, for two reasons:
   (a) There are two ways to assign a 0 or 1 to an edge of the tree. In Example 3.3, we have chosen to assign 0 to the left edge and 1 for the right. However, it is possible to assign 0 to the right and 1 to the left. This would not make any difference to the compression ratio.
   (b) There are a number of different ways to insert a combined item into the frequency table. This leads to different binary trees. We have chosen in the same example to:
      i. make the item at the higher position the left child
      ii. insert the combined item on the frequency table at the highest possible position.
2. The Huffman tree built using our approach in the example tends to be more balanced than those built using other approaches. The code derived with our method in the example is called *canonical minimum-variance* Huffman code.

   The differences among the lengths of codewords in a canonical minimum-variance code turn out to be the minimum possible.
3. The frequency table can be replaced by a probability table. In fact, it can be replaced by any approximate statistical data at the cost of losing some compression ratio. For example, we can apply a probability table derived from a typical text file in English to any source data.
4. When the alphabet is small, a fixed length (less than 8 bits) code can also be used to save bits.

   Example 3.5 *If the size of the alphabet set is not bigger than 32*[3]*, we can use five bits to code each character. This would give a saving percentage of*

$$\frac{8 \times 32 - 5 \times 32}{8 \times 32} = 37.5\%.$$

[3] *i.e. the size is smaller or equal to 32.*

Shannon-Fano coding

This is another approach very similar to Huffman coding. In fact, it is the first well-known coding method. It was proposed by C. Shannon (Bell Labs) and R. M. Fano (MIT) in 1940.

The Shannon-Fano coding algorithm also uses the probability of each symbol's occurrence to construct a code in which each codeword can be of different length. Codes for symbols with low probabilities are assigned more bits, and the codewords of various lengths can be uniquely decoded.

Shannon-Fano algorithm

Given a list of symbols, the algorithm involves the following steps:

1. Develop a frequency (or probability) table

2. Sort the table according to frequency (the most frequent one at the top)
3. Divide the table into 2 halves with similar frequency counts
4. Assign the upper half of the list a *0* and the lower half a *1*
5. Recursively apply the step of division (2.) and assignment (3.) to the two halves, subdividing groups and adding bits to the codewords until each symbol has become a corresponding leaf on the tree.

Example 3.6 *Suppose the sorted frequency table below is drawn from a source. Derive the Shannon-Fano code.*

Symbol	Frequency
A	15
B	7
C	6
D	6
E	5

Solution

1. First division:

   (a) Divide the table into two halves so the sum of the frequencies of each half are as close as possible.

Symbol	Frequency	
A	15	22
B	7	
1111111111111111111111111	First division	
C	6	17
D	6	
E	5	

   (b) Assign one bit of the symbol (e.g. upper group 0s and the lower 1s).

Symbol	Frequency	Code
A	15	0
B	7	0
1111111111111111111111111	First division	
C	6	1
D	6	1
E	5	1

2. Second division:
   Repeat the above recursively to each group.

```
Symbol          Frequency       Code
-----------------------------------
A               15              00
22222222222222222222222222222   Second   division
B               7               01
11111111111111111111111111111   First  division
C               6               10
22222222222222222222222222222   Second   division
D               6               11
E               5               11
```

3. Third division:

```
Symbol          Frequency       Code
-----------------------------------
A               15              00
22222222222222222222222222222   Second   division
B               7               01
11111111111111111111111111111   First  division
C               6               10
22222222222222222222222222222   Second   division
D               6               110
33333333333333333333333333333   Third   division
E               5               111
```

4. So we have the following code (consisting of 5 codewords) when the recursive process ends:

```
A    B    C    D    E
00   01   10   110  111
```

Observation on Shannon-Fano coding

1. It is not always easy to find the best division (see Step 3., the encoding algorithm).

2. The encoding process starts by assigning the most significant bits to each code and then works down the tree recursively until finished. It is as to construct a binary tree from the top. The following tree is for the example above:

```
        ABCDE
       0/   \1
      AB     CDE
   0/ \1   0/   \1
   A   B   C    DE
              0/   \1
              D     E
```

3. The decoding process is similar to Huffman decoding but the encoding process is not as simple and elegant as Huffman encoding.

Learning outcomes

On completion of your studies in this chapter, you should be able to:

- describe Huffman coding and Shannon-Fano coding
- explain why it is not always easy to implement Shannon-Fano algorithm
- demonstrate the encoding and decoding process of Huffman and Shannon-Fano coding with examples.

Activities

1. Derive a Huffman code for string AAABEDBBTGGG.
2. Derive a Shannon-Fano code for the same string.
3. Provide an example to show step by step how Huffman decoding algorithm works.
4. Provide a similar example for the Shannon-Fano decoding algorithm.

Laboratory

1. Derive a simple version of Huffman algorithm in pseudocode.
2. Implement your version of Huffman algorithm in Java (or C, C++).
3. Similar to Lab2, provide two source files: the good and the bad. Explain what you mean by good or bad.
4. Implement the Shannon-Fano algorithm.
5. Comment on the difference the Shannon-Fano and the Huffman algorithm.

Sample examination questions

1. Derive step by step a canonical minimum-variance Huffman code for alphabet *{*A, B, C, D, E, F*}*, given that the probabilities that each character occurs in all messages are as follows:

Symbol	Probability
A	0.3
B	0.2
C	0.2
D	0.1
E	0.1
F	0.1

2. Compute the average length of the Huffman code derived from the above question.

Given $S = \{A, B, C, D, E, F, G, H\}$ and the symbols' occurring probabilities 0.25, 0.2, 0.2, 0.18, 0.09, 0.05, 0.02, 0.01, construct a canonical minimum-variance